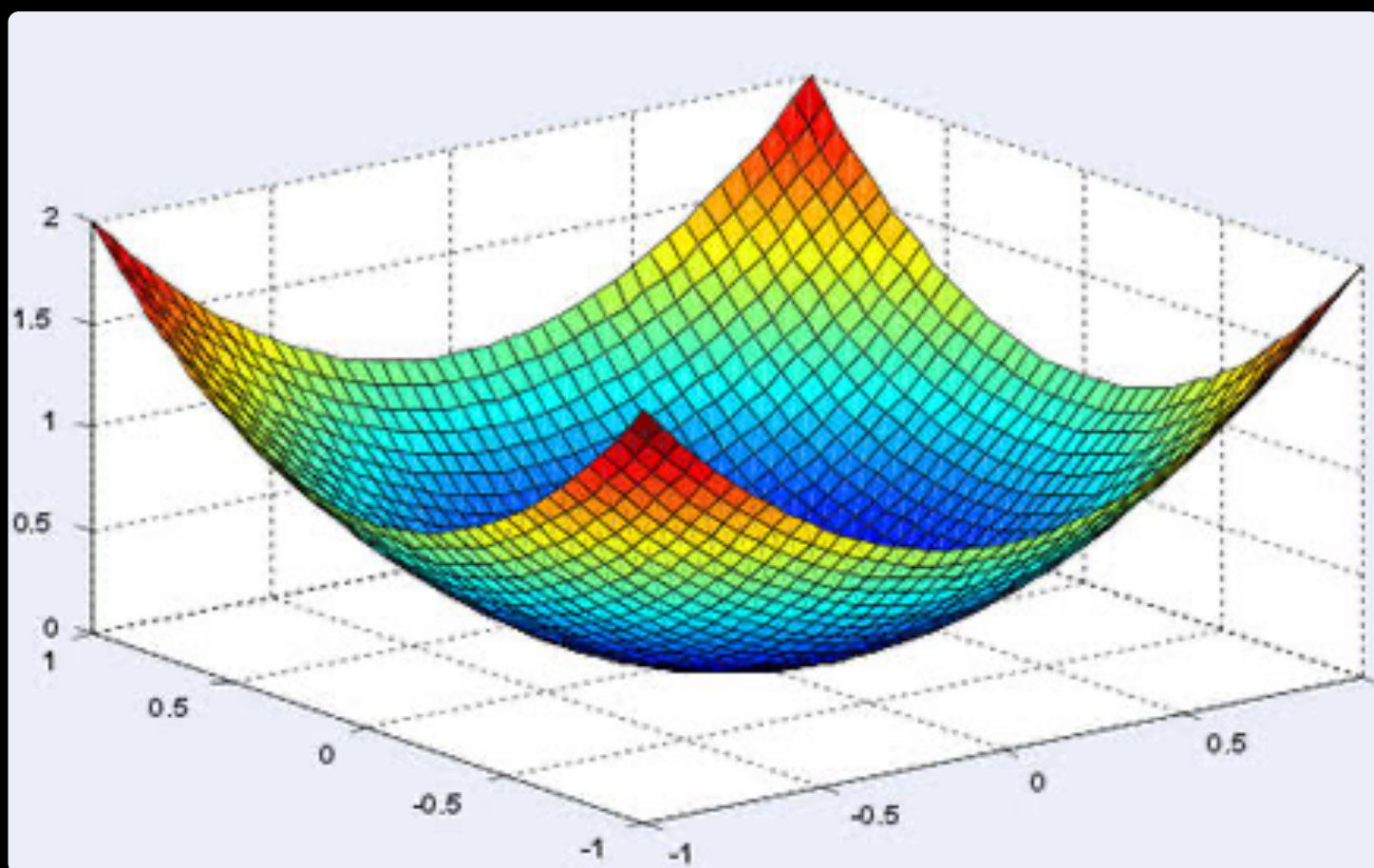
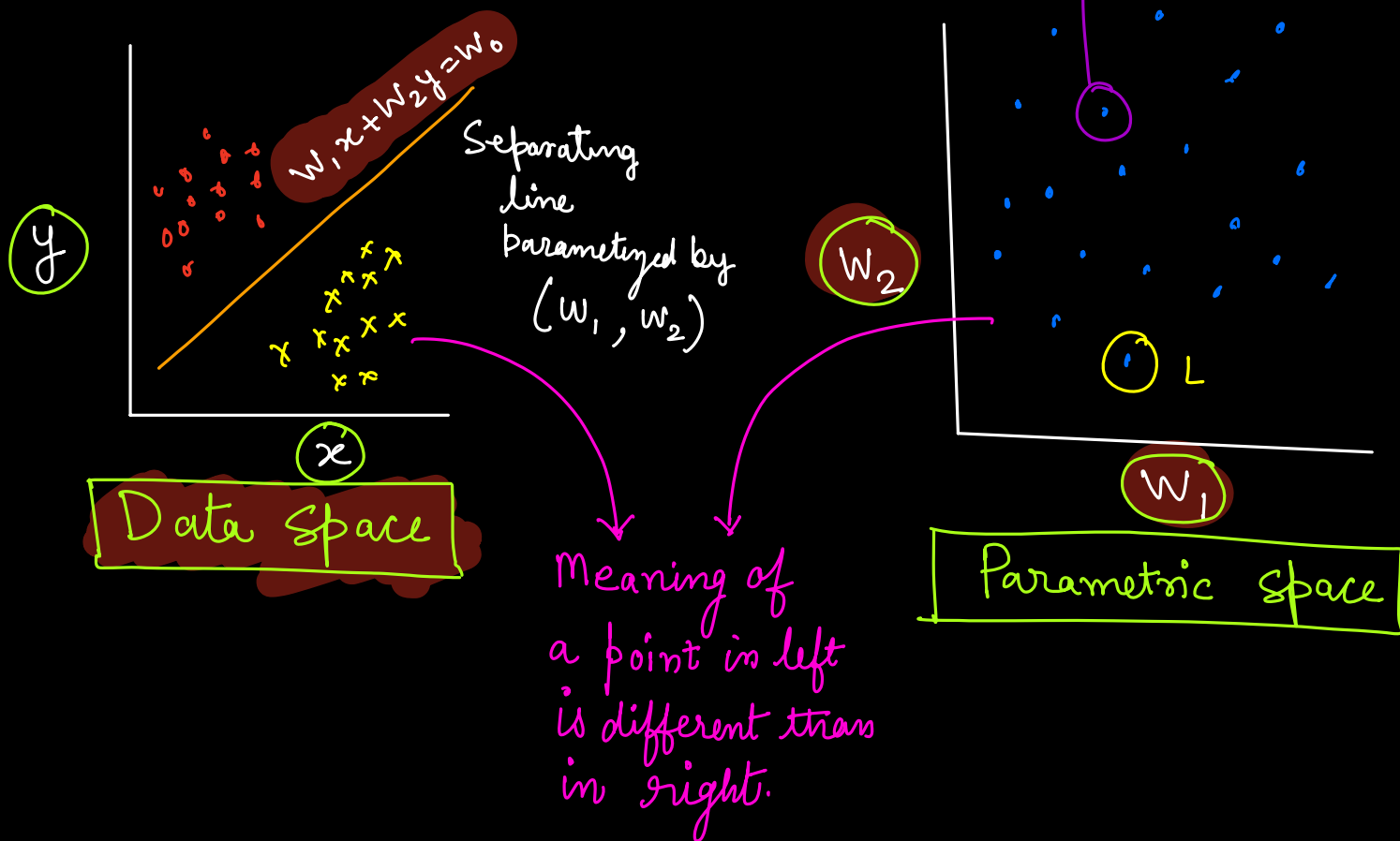
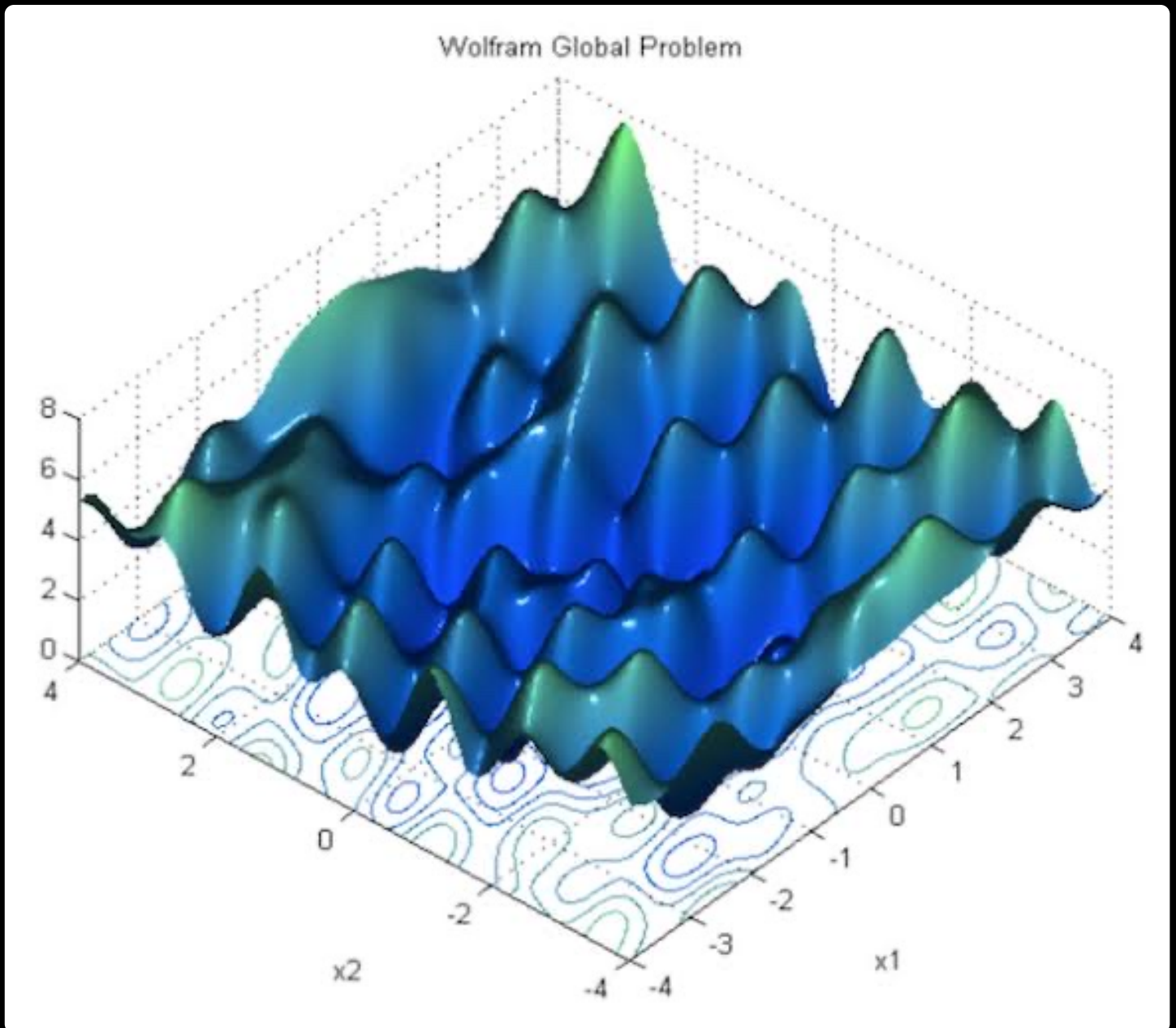


Let us assume 2-D datapoints.



Usually we do not get such nice
Convex error surface. There can
be so many maxima's & minima's
and often optimization got stuck in local
extremas.



That is why we need a "good" optimizer.

Our objective:

Exploring the error surface efficiently and get the global minima as soon as possible.

Data

$$\forall \{x_i, y_i\} \\ i=1 \text{ to } 1000$$

Parameter initialization

$$W = [w_1, w_2]$$



we need to know
an efficient update
rule

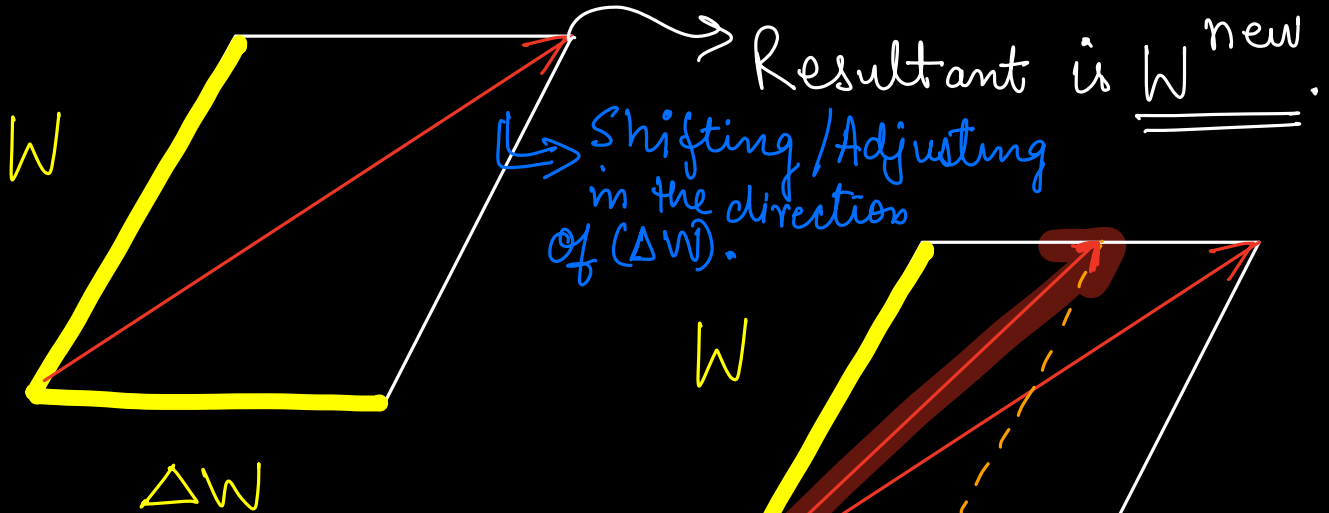


$$\Delta W = [\Delta w_1, \Delta w_2]$$

(But how to update)

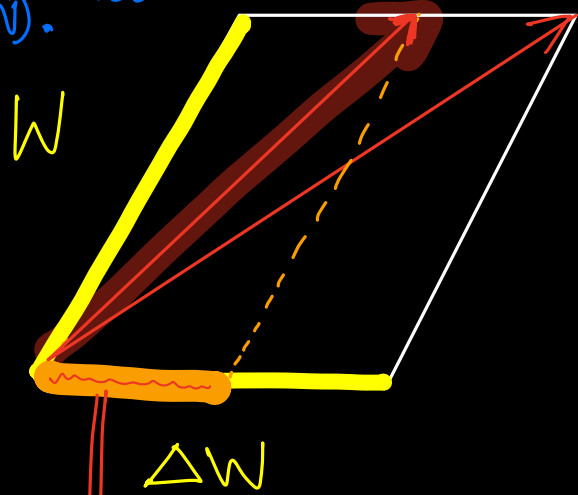
$$\text{new } W = W + \Delta W$$

Update rule



$$\text{new } W = W + \eta \Delta W$$

How to estimate the best (ΔW) updation.



$\eta \Delta W$ $\{ \eta \text{ between } (0, 1) \}$
(moving cautiously)

Assuming that our loss function is L .
Then L must be a function of (W) .

$$L(W) \equiv \text{old loss}$$

$$L(W + \eta \Delta W) = \text{new loss}$$

Our requirement is :

$$L(W + \eta \Delta W) - L(W) < 0$$

Let us apply Taylor series :

$$L(W + \eta \Delta W) = L(W) + \eta (\Delta W)^T \nabla_W (L(W)) \\ + \frac{\eta^2}{2!} * (\Delta W)^T \nabla^2 (L(W)) (\Delta W) \\ + \dots$$

η is very small $\Rightarrow$ (All high order terms $\rightarrow 0$)

$$L(w + \eta \Delta w) = L(w) + \eta (\Delta w)^T \nabla_w (L(w))$$

$$L(w + \eta \Delta w) - L(w) = \eta (\Delta w)^T \nabla_w (L(w))$$

This implies:

$$(\Delta w)^T \nabla_w (L(w)) < 0$$

$$(1 \times 2) \quad (2 \times 1)$$

$$\boxed{} * \boxed{} \Rightarrow \text{getting a scalar value.}$$

This is a dot product between (2) vectors

$$(\Delta w)^T \quad \& \quad (\nabla_w (L(w))) \quad \left(\cos \theta = \frac{a \cdot b}{|a| |b|} \right)$$

$\Rightarrow$ Assuming the angle between them is " θ ", above scalar is just the $\boxed{\cos(\theta)}$
hence always lies between $\boxed{[-1, 1]}$

$$\left(\cos \theta = \frac{a \cdot b}{|a| |b|} \right)$$

$$-1 \leq \cos(\theta) \leq 1$$

$$-1 \leq \frac{(\Delta w^T) * (\nabla_w (L(w)))}{\text{Some (+ve) value (say } P)} \leq 1$$

$$-P \leq P * \cos(\theta) \leq P$$

$$-P \leq (\Delta w^T) * (\nabla_w (L(w))) \leq P$$

$$-P \leq \underbrace{L(w + \eta \Delta w) - L(w)}_{\text{this value to be most (-ve) negative.}} \leq P$$

When can we have this value to be most (-ve) negative.

Answer is :

When $\cos(\theta)$ is $-1 \Rightarrow \theta = 180^\circ$

And the value is $-P$

(θ) is the angle between

(ΔW) and $\left(\frac{\partial L}{\partial W}\right)$ derivative of loss w.r.t (W) .

$(\theta \equiv 180^\circ)$

$\Rightarrow$ Implies that we need to move in the Opposite direction of $\left(\frac{\partial L}{\partial W}\right)$

$$\therefore \left[\Delta W = -\eta \frac{\partial L}{\partial W} \right]$$

$$\therefore \left[W^{\text{new}} = W - \eta \frac{\partial \mathcal{L}}{\partial W} \right] \text{ (GD)}$$

(A) Gradient Descent update Rule

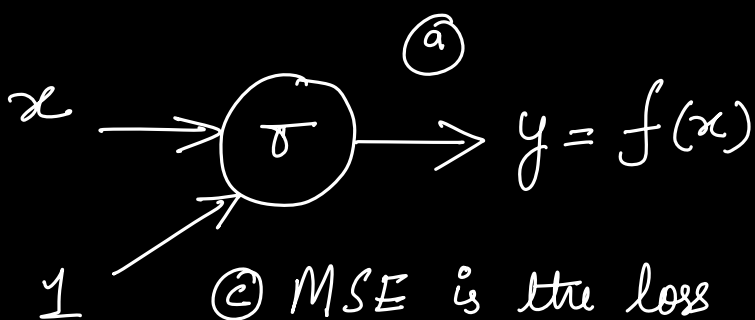
$$(*) \quad w_{t+1} = w_t - \eta \nabla w_t$$

$$b_{t+1} = b_t - \eta \nabla b_t$$

Here $\nabla w_t = \frac{\partial \mathcal{L}(w, b)}{\partial w} \text{ at } (w=w_t, b=b_t)$

$$\nabla b_t = \frac{\partial \mathcal{L}(w, b)}{\partial b} \text{ at } (w=w_t, b=b_t)$$

This is for 1D input Problem with 1 neuron (Sigmoid).



$$f(x) = \frac{1}{1 + e^{-(wx+b)}} \quad \text{(b)}$$

(c) MSE is the loss

$$\mathcal{L}(w, b) = \frac{1}{2} * (f(x) - y)^2$$

$$\nabla w = \frac{\partial \mathcal{L}(w, b)}{\partial w} = \frac{\partial}{\partial w} \left(\frac{1}{2} * (f(x) - y)^2 \right)$$

$$= (f(x) - y) * f(x) * (1 - f(x)) * x$$

Hence

$$\nabla w_i = \sum_{i=1}^N \left[(f(x_i) - y_i) * f(x_i) * (1 - f(x_i)) * x_i \right]$$

$$\nabla b_i = \sum_{i=1}^N \left[(f(x_i) - y_i) * f(x_i) * (1 - f(x_i)) \right]$$

⊛ Steep Gradient or High Gradient

↳ More update, fast movement

⊛ Gentle Gradient or Low Gradient

↳ Less update, slow movement.

{ GD takes time to navigate through the regions having gentle slope. }

Ⓑ Momentum Based Gradient Descent.

$$\begin{aligned} \text{⊛ } \text{update}_t &= \underbrace{\gamma \cdot \text{update}_{t-1}}_{\text{history}} + \underbrace{\eta \nabla W_t}_{\text{Current Update}} \\ w_{t+1} &= w_t - \text{update}_t \end{aligned}$$

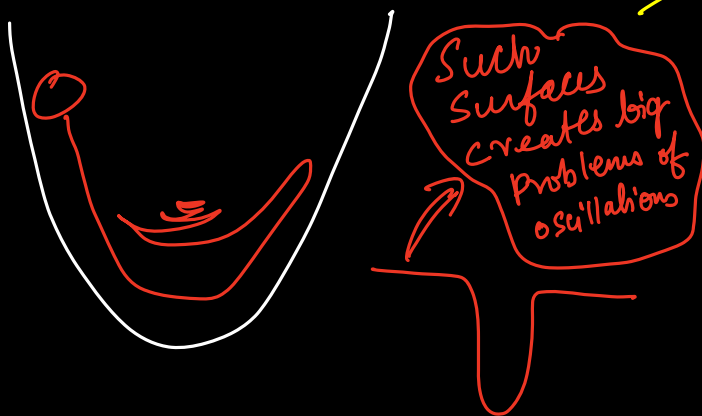
Consider current update along with history (momentum)

Just see that

$$\text{update}_4 = \gamma^3 \eta \nabla w_1 + \gamma^2 \eta \nabla w_2 + \gamma \eta \nabla w_3 + \eta \nabla w_4$$

This will help you
to move faster even in
Gentle slope region once the
momentum got built.

(Geometrically
weighted update
history accumulation)



This will also let you
to move fast even in
steeper slope. It cause
you to miss the minima
and (longer U-turns)

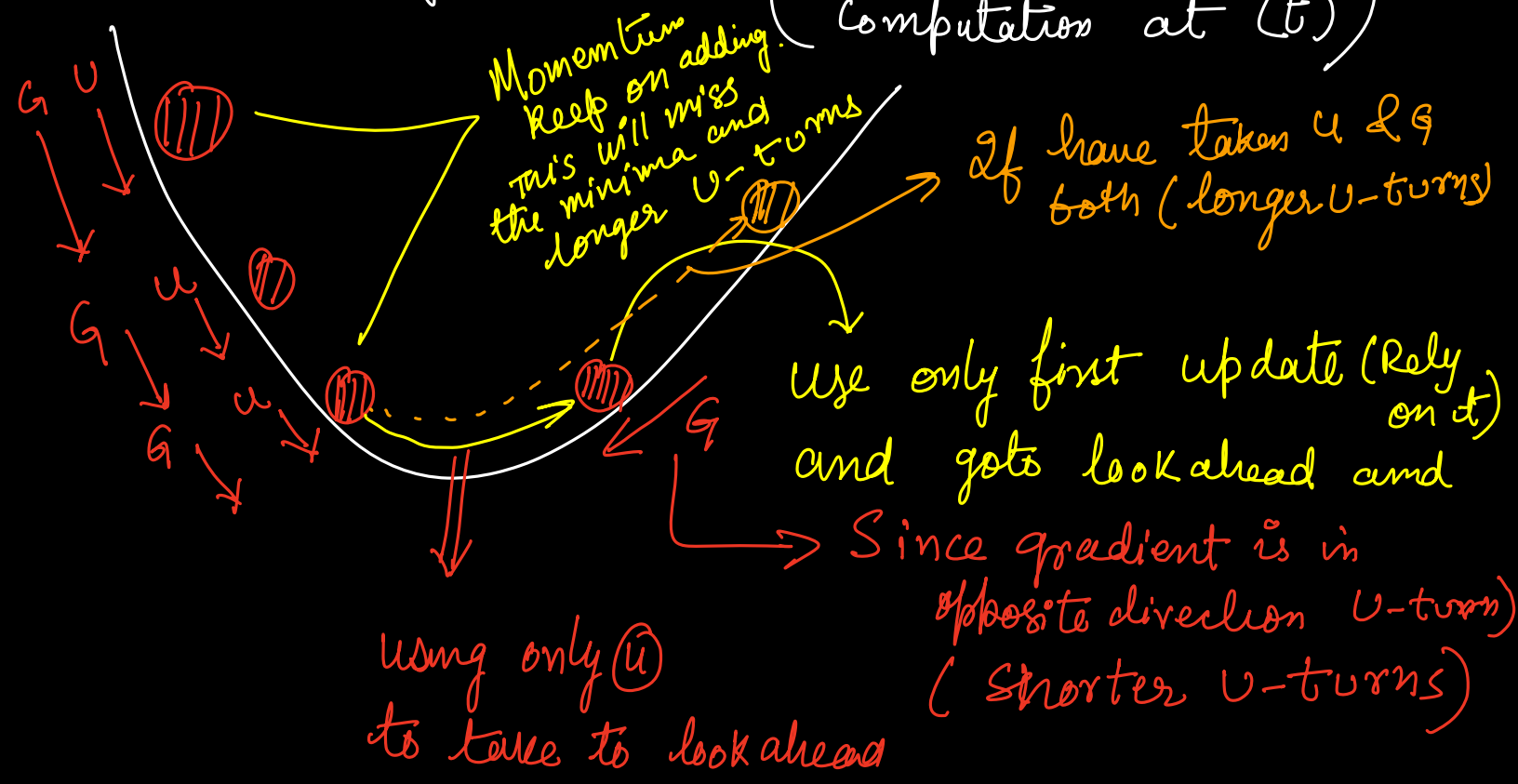
③ Nesterov accelerated gradient (NAG)
(look before you leap)

$$w_{\text{look-ahead}} = w_t - \gamma \text{update}_{t-1}$$

$$\text{update}_t = \gamma \text{update}_{t-1} + \eta \nabla w_{\text{look-ahead}}$$

$$w_{t+1} = w_t - \text{update}_t$$

- (*) first look ahead in time using/relying your historical updates and then at the point (look-ahead), compute the gradient to actually take step (instead of gradient computation at t)



(D) Ada Grad (Adaptive (learning rate) Gradient descent.)

we already have seen that ^{weights} gradients depends on the corresponding input dimensions when loss is MSE.

$$\nabla W_i = (f(x) - y) * f(x) * (1 - f(x)) * x_i$$

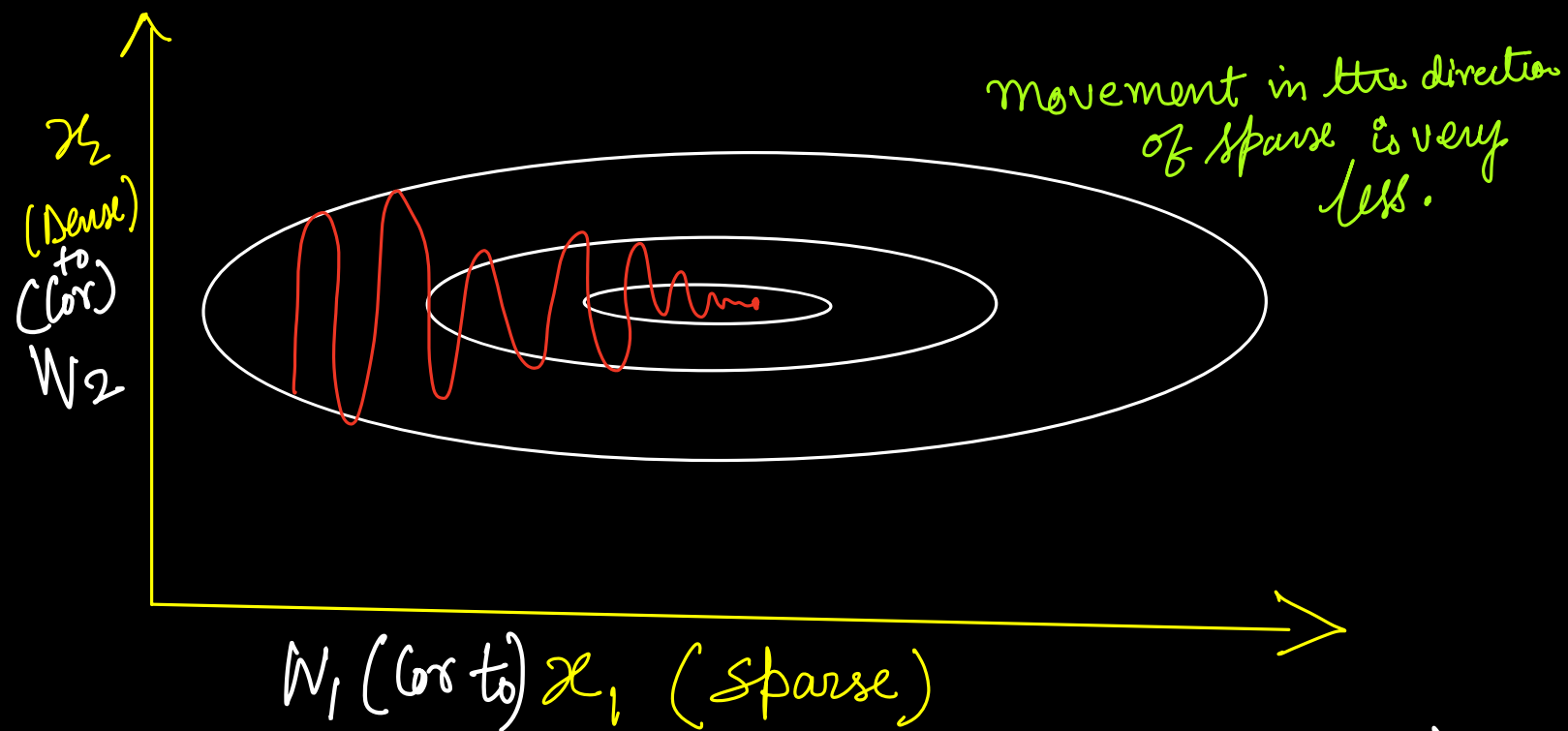
hence update also depends upon input value.

Assuming that Some input Say (x_k) is very sparse. ($x_k =$ Are you physically disabled)
 $x_k =$ Mentally unfit (Majority of time it's zero)
 $x_k =$ Criminal record
 $x_k =$ Gone to jail

Sparse but important

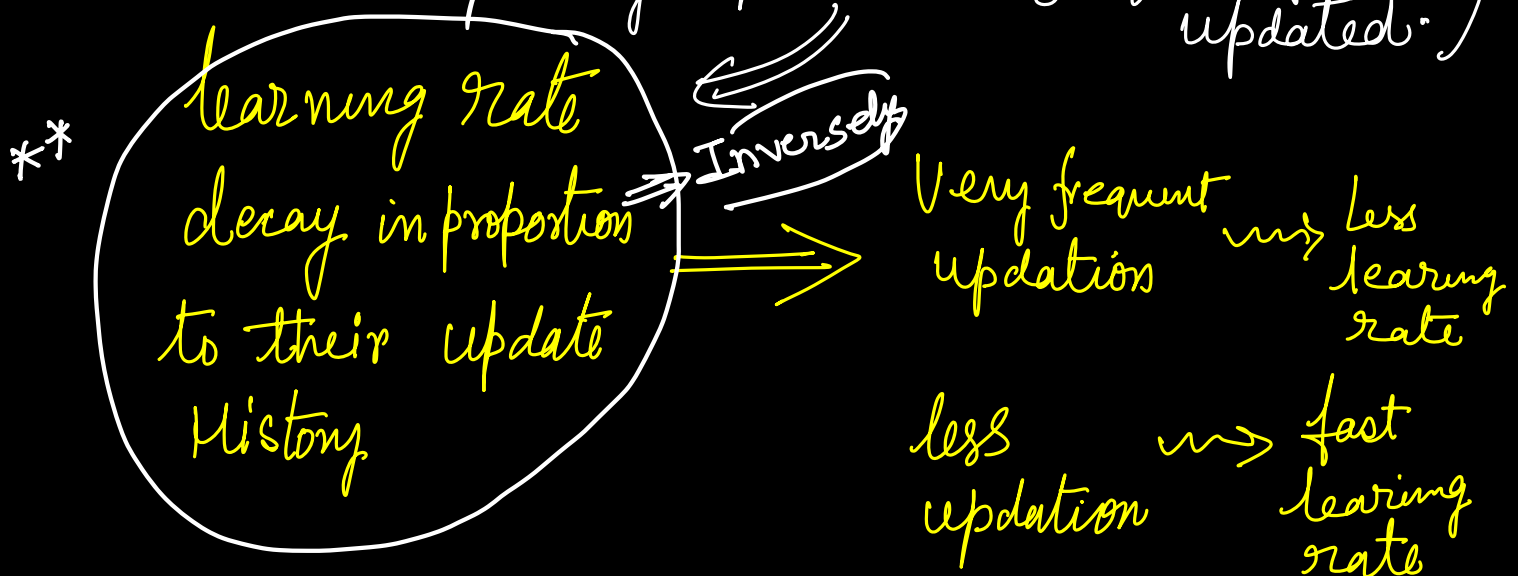
(*) Such sparse input will let its corresponding weight to be updated very less frequently.

$\Rightarrow$ Opposite to other weights (W_k) will receive very less weight updates and other dense features weights will get many updates.



(Such sparsity is very common in NN with 1000's of Parameters.)

(*) We require different learning rate for different Parameters (depending upon how frequently it is updated.)



Momentum based learning rate decay.

momentum of gradient only $\rightsquigarrow$

$$v_t = v_{t-1} + (\nabla w_t)^2$$

(the ∇ -ve update should not be cancelled)

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} * (\nabla w_t)$$

The effective learning rate $\left(\frac{\eta}{\sqrt{v_t + \epsilon}} \right)$ is maintained. $\left(\frac{\eta}{\sqrt{v_t}} \right)$ is even better but w/o much explanation.

AdaGrad reduces the learning rate of frequent updated parameters very fast. Hence soon they keep on receiving very small updates and don't make much progress close to convergence.

AdaGrad $\Rightarrow$ Decay learning rate maybe too aggressively.

(E) RMS Prop

Decay the denominator of effective learning rate: $\frac{\eta}{\sqrt{v_t + \epsilon}}$
 (avoiding its fast growth) $\leftarrow$ rate

$$\beta = 0.9$$

Just Controlling v_t is be growing very large (aggressive)
 (keep on accumulating but in a weighted manner)

$$v_t = \beta * v_{t-1} + (1 - \beta) (\nabla w_t)^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} * \nabla w_t$$

$$\beta = 0.9 \mid 1 - \beta = 0.1$$

more weight to history $\rightarrow$ history loss with β
 less weight to current update $\beta(1-\beta)$

$$v_1 = (0.1) (\nabla w_1)^2 \quad v_2 = (0.9)(0.1) (\nabla w_1)^2 + (0.1) (\nabla w_2)^2$$

$$V_3 = (0.9)^2 (0.1) (\nabla w_1)^2 + (0.9) (0.1) (\nabla w_2)^2 + (0.1) (\nabla w_3)^2$$

$$V_4 = (0.9)^3 (0.1) (\nabla w_1)^2 + (0.9)^2 (0.1) (\nabla w_2)^2 + (0.9) (0.1) (\nabla w_3)^2 + (0.1) (\nabla w_4)^2$$

(0.0729) (0.081) (0.09)

$$= (0.07) (\nabla w_1)^2 + (0.081) (\nabla w_2)^2 + (0.09) (\nabla w_3)^2 + (0.1) (\nabla w_4)^2$$

Geometrically Annihilation

Exponentially weighted averaging

⇒ This was with learning rate and its scaling. Some times increasing or decreasing (depending upon update history)

Ⓕ ADAM: [Adaptive Moments]
(Adaptation via. moments)

⊗ RMS Prop + Cumulative history of gradients (momentum)

(Cumulative history of gradients square)

↪ Second moment

↪ first moment

$$(i) \quad m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla W_t$$

$$(ii) \quad v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla W_t)^2$$

$$(iii) \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \& \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$(iv) \quad W_{t+1} = W_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} * \hat{m}_t$$

Bias
Correction
(Scaling
of m_t &
 v_t)

Suggested by
RMS-Prop

(Learning rate
Scaling by accumulated
history of 2nd Momentum)

(Handling learning
rate)

This need to be
 ∇W_t (But instead
we are using accumulated
history. (Mom-based
G.D.)

(Handling/using
first momentum
and its history
for updation as
Suggested by G.D.)

Momentum
base

ADAM $\Rightarrow$ Works very well practically.

(*) $\beta_1 = 0.9$ and $\beta_2 = 0.999$
with $\epsilon = 1e-8$

(*) $\eta = 0.001, 0.0001$
 $10^{-3}, 10^{-4}$

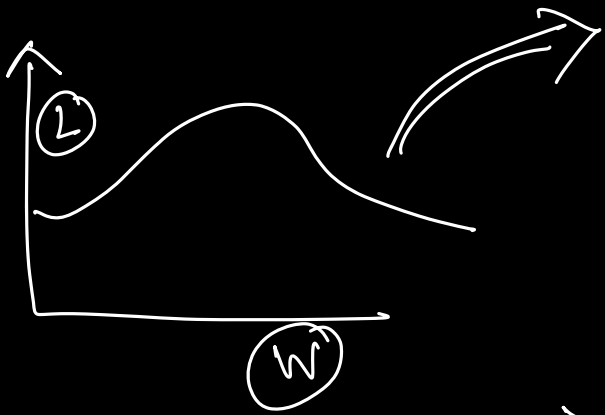
depends on amount of data
Batch size

(Also $\Rightarrow$ SGD with momentum and learning rate decay performs also well)

Use ADAM if not clear to tweak/
Play with (M-SGD + DLR)

(Proving for bias correction)

Let us assume we are updating Stochastically
i.e w.r.t a particular point (x_t) at time (t).



Depending upon the error Surface (and its structure) the gradient at each point w.r.t the weight vector will vary.

$\Rightarrow$ Assuming (g_t) or (∇W_t) are coming from a distribution being modelled by some Random Variable.

Each point can update W 's as $\frac{\partial L}{\partial W_t}$ or (∇W_t) or (g_t)



(a)

Wishes to take the weight vectors "good" for its own loss reduction.

(b)

But we need to ensure that updates should not be very big and takes us to somewhere which is really off the solution.

(Hence we have used Exponentially moving averages)

Basically we do not want to update with more than the mean of the (∇W_t) 's distribution.

- ⊗ Not relying too much on current gradient but rely on overall behaviour of gradients over many time steps.

We are interested to move with via.

expected value of updates $E[m_t]$
Average of update.

[This contains gradients at time t as well as history.]

(we don't want history to blow up the updates)

Ideally we want

$$E[m_t] = E[\nabla w_t] = E[g_t]$$

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * g_t$$

$$m_0 = 0$$

$$m_1 = \beta m_0 + (1 - \beta) g_1 = (1 - \beta) g_1$$

$$m_2 = \beta m_1 + (1 - \beta) g_2 = \beta(1 - \beta) g_1 + (1 - \beta) g_2$$

$$m_3 = (1 - \beta) \sum_{i=1}^3 \beta^{3-i} g_i$$

↓
In general

$$m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$$

$$E[m_t] = E\left[(1 - \beta) * \sum_{i=1}^t \beta^{t-i} g_i\right]$$

$$E[m_t] = (1 - \beta) E\left[\sum_{i=1}^t \beta^{t-i} g_i\right]$$

(Expectation)
of Sum
↔
Sum of expectations

$$= (1 - \beta) \sum_{i=1}^t E[\beta^{t-i} g_i]$$

↪ [Constant for that i]

$$= (1 - \beta) \sum_{i=1}^t \beta^{t-i} E[g_i]$$

(↓
 $E(g_1) + E(g_2) \dots$)

But as we have assumed that all gradients are coming from same distribution (or following q distribution)

$$\therefore \forall \sum_{i=1}^t E[g_i] = E[g] \quad \xrightarrow{\text{(const)}} \text{Mean of the global distribution}$$

Hence

$$E[m_t] = (1-\beta) \sum_{i=1}^t (\beta)^{t-i} E[g]$$

$$= E[g] (1-\beta) \sum_{i=1}^t \beta^{t-i}$$

$$= E[g] (1-\beta) (\beta^{t-1} + \beta^{t-2} + \dots + \beta^0)$$

$$= E[g] \cancel{(1-\beta)} \frac{(1-\beta^t)}{\cancel{(1-\beta)}}$$

$$= E[g] (1-\beta^t) \quad \text{(again const)}$$

$$\therefore \boxed{E\left[\frac{m_t}{1-\beta^t}\right] = E[g]} \quad \rightarrow \quad E[\hat{m}_t] = E[g]$$

Bias corrected $(m_t) \rightarrow \hat{m}_t$ follows